



Pasión por la Tecnología

Fundamentos de Gestión de Datos

Docente:
Pilar Rocío Sayán Mejía

LABORATORIO N° 03

Semana 3:
SQL Avanzado - Joins y Agrupaciones



LABORATORIO DIRIGIDO N° 03	
Tema: SQL Avanzado - Joins y Agrupaciones	
DOCENTE: Pilar Rocío Sayán Mejía	CURSO: Fundamentos de Gestión de Datos

I. Elemento de la capacidad terminal de la semana

Aplica consultas SQL avanzadas utilizando JOINS (INNER, LEFT, RIGHT, FULL OUTER), agrupaciones con GROUP BY y HAVING, subconsultas y vistas en SQLite mediante Google Colab para analizar relaciones entre múltiples tablas de una base de datos de ventas.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red sin supervisión.
- Prohibida la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en el lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.
- No descargar software no autorizado en los equipos del laboratorio.
- Todo código se ejecuta únicamente en Google Colab; no instalar paquetes en la máquina local.

III. Fundamento teórico

Antes de realizar este laboratorio, el estudiante debe haber revisado el siguiente material:

- Diapositivas de la Semana 3: SQL Avanzado - Joins y Agrupaciones.
- Date, C. J. (2003). An Introduction to Database Systems. Addison-Wesley. Capítulo 5.
- Beaulieu, A. (2020). Learning SQL. O'Reilly Media. Capítulos 3-5.
- Documentación de SQLite: https://www.sqlite.org/lang_select.html

Conceptos clave:

JOIN (INNER, LEFT, RIGHT, FULL OUTER), GROUP BY, HAVING, subconsultas, vistas (VIEW), alias, integridad referencial, clave foránea, agregación (COUNT, SUM, AVG, MAX, MIN), optimización de consultas.

IV. Recursos

- Computadora con acceso a internet y navegador web.
- Cuenta de Google para acceder a Google Colab.
- Base de datos SQLite: ventas.db (tablas: clientes, pedidos, productos, detalles_pedido).
- Librerías de Python: pandas, sqlite3 (preinstaladas en Colab).

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.
- Duración estimada: 2 horas (1 sesión de laboratorio).
- Entregable: Notebook ejecutado de Google Colab (.ipynb) con código ejecutado y respuestas completas.

VI. Procedimiento

Actividad 1: Revisión de conceptos - SQL Avanzado

Complete la siguiente tabla con definiciones propias basadas en lo estudiado en la clase teórica. No copie textualmente de los materiales; use sus propias palabras.

Concepto / Principio	Definición con sus propias palabras
1. ¿Qué es un JOIN y para qué se utiliza en SQL?	
2. Diferencia entre INNER JOIN y LEFT JOIN	
3. ¿Qué es RIGHT JOIN y FULL OUTER JOIN?	
4. ¿Qué hace la cláusula GROUP BY?	
5. Diferencia entre WHERE y HAVING	
6. ¿Qué es una subconsulta y cuándo se usa?	
7. ¿Qué es una vista (VIEW) y sus ventajas?	
8. Funciones de agregación: COUNT, SUM, AVG, MAX, MIN	
9. ¿Qué es un alias en SQL y para qué sirve?	

10. Buenas prácticas de legibilidad en consultas SQL	
--	--

Actividad 2: Desarrollo práctico - Análisis de Ventas con JOINS

En esta actividad trabajarás con una base de datos de ventas que contiene 4 tablas relacionadas: clientes, pedidos, productos y detalles_pedido. Aplicarás diferentes tipos de JOINS, agrupaciones y subconsultas. Abre un nuevo notebook en Google Colab y sigue los pasos indicados.

Paso 1: Conexión a la base de datos SQLite

Ejecuta el siguiente código para conectarte a la base de datos y explorar las tablas:

```
import sqlite3
import pandas as pd
# Conexión a la base de datos
conn = sqlite3.connect('ventas.db')
# Ver todas las tablas
query = "SELECT name FROM sqlite_master WHERE type='table';"
tablas = pd.read_sql_query(query, conn)
print(tablas)
```

Pregunta 1: ¿Cuáles son las tablas de la base de datos? ¿Qué columnas tiene cada tabla? Usa PRAGMA table_info(nombre_tabla) para explorar.

Respuesta: _____

Paso 2: INNER JOIN - Clientes con pedidos

Ejecuta el siguiente código para relacionar clientes con sus pedidos:

```
query = """
SELECT c.nombre, c.ciudad, p.id_pedido, p.fecha, p.total
FROM clientes c
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente
ORDER BY p.fecha DESC
LIMIT 10;
"""
df = pd.read_sql_query(query, conn)
print(df)
```

Pregunta 2: ¿Cuántos clientes tienen al menos un pedido? ¿Qué ocurre con los clientes que no tienen pedidos en un INNER JOIN?

Respuesta:

Paso 3: LEFT JOIN - Incluir clientes sin pedidos

Ejecuta el siguiente código para ver todos los clientes, incluyendo los que no tienen pedidos:

```
query = ""
SELECT c.nombre, c.ciudad, p.id_pedido, p.total
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
ORDER BY c.nombre;
""

df_left = pd.read_sql_query(query, conn)
print(df_left.head(20))
```

Pregunta 3: ¿Cuál es la diferencia entre el resultado del INNER JOIN y el LEFT JOIN? ¿Cuántos clientes no tienen pedidos?

Respuesta:

Paso 4: GROUP BY - Total de ventas por cliente

Ejecuta el siguiente código para calcular el total de ventas por cliente:

```
query = ""
SELECT c.nombre, COUNT(p.id_pedido) as num_pedidos,
       SUM(p.total) as total_comprado
FROM clientes c
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente
ORDER BY total_comprado DESC;
""

df_ventas = pd.read_sql_query(query, conn)
print(df_ventas)
```

Pregunta 4: ¿Cuáles son los 5 clientes con mayor monto total de compras? ¿Cuál es el promedio de pedidos por cliente?

Respuesta:

Paso 5: HAVING - Filtrar grupos

Ejecuta el siguiente código para filtrar clientes con más de 2 pedidos:

```
query = ""
```

```
SELECT c.nombre, COUNT(p.id_pedido) as num_pedidos,
       SUM(p.total) as total_comprado
```

```
FROM clientes c
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente
HAVING COUNT(p.id_pedido) > 2
ORDER BY total_comprado DESC;
'''

df_having = pd.read_sql_query(query, conn)
print(df_having)
```

Pregunta 5: ¿Cuál es la diferencia entre usar WHERE y HAVING? ¿Por qué no se puede usar WHERE en este caso?

Respuesta:

Paso 6: JOIN múltiples tablas - Productos más vendidos

Ejecuta el siguiente código para analizar los productos más vendidos:

```
query = '''
SELECT pr.nombre_producto, pr.categoria,
       SUM(d.cantidad) as total_vendido,
       SUM(d.cantidad * d.precio_unitario) as ingresos
FROM productos pr
INNER JOIN detalles_pedido d ON pr.id_producto = d.id_producto
INNER JOIN pedidos p ON d.id_pedido = p.id_pedido
GROUP BY pr.id_producto
ORDER BY ingresos DESC
LIMIT 10;
'''

df_productos = pd.read_sql_query(query, conn)
print(df_productos)
```

Pregunta 6: ¿Cuáles son los 3 productos que generan más ingresos? ¿A qué categoría pertenecen?

Respuesta:

Paso 7: Subconsultas - Clientes above del promedio

Ejecuta el siguiente código para encontrar clientes con compras above del promedio:

```
query = '''
SELECT c.nombre, SUM(p.total) as total_comprado
FROM clientes c
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente
```

```
GROUP BY c.id_cliente
HAVING SUM(p.total) > (SELECT AVG(total) FROM pedidos)
ORDER BY total_comprado DESC;
'''
df_subconsulta = pd.read_sql_query(query, conn)
print(df_subconsulta)
```

Pregunta 7: ¿Cuál es el promedio de ventas por pedido? ¿Cuántos clientes superan ese promedio?

Respuesta:

Paso 8: Crear una vista (VIEW)

Ejecuta el siguiente código para crear una vista de resumen de ventas:

```
query = '''
CREATE VIEW IF NOT EXISTS vw_resumen_ventas AS
SELECT c.nombre, c.ciudad,
       COUNT(p.id_pedido) as num_pedidos,
       SUM(p.total) as total_comprado,
       AVG(p.total) as ticket_promedio
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente;
'''
conn.execute(query)
print('Vista creada exitosamente')
# Consultar la vista
df_vista = pd.read_sql_query('SELECT * FROM vw_resumen_ventas;', conn)
print(df_vista)
```

Pregunta 8: ¿Qué ventajas tiene usar una vista en lugar de escribir la consulta completa cada vez?

Respuesta:

Actividad 3: Caso de estudio - Top 5 Clientes

Contexto: El gerente comercial de la empresa necesita un informe con los 5 clientes más valiosos para ofrecerles beneficios exclusivos. Tu tarea es crear una consulta que identifique a estos clientes y justificar el análisis.

Responde las siguientes preguntas en celdas de texto (Markdown) de tu notebook de Colab:

Pregunta A: Escribe una consulta SQL que identifique el TOP 5 de clientes por monto total de compras. La consulta debe incluir: nombre del cliente, ciudad, número de pedidos, monto total y ticket promedio.

Respuesta:

Pregunta B: ¿Qué tipo de JOIN utilizaste en la consulta anterior? ¿Por qué es el más adecuado para este caso?

Respuesta:

Pregunta C: Identifica qué clientes no han realizado ninguna compra. ¿Qué tipo de JOIN necesitas para encontrarlos?

Respuesta:

Pregunta D: Si quisieras analizar las ventas por categoría de producto, ¿qué tablas necesitarías relacionar? Escribe la consulta.

Respuesta:

Pregunta E: Crea una vista llamada vw_ventas_por_ciudad que muestre el total de ventas agrupado por ciudad. ¿Qué insights puedes obtener de esta vista?

Respuesta:

Conclusiones

Escriba las conclusiones técnicas y de aprendizaje obtenidas tras la práctica:

1.

2.

3.

Material complementario

1. Beaulieu, A. (2020). Learning SQL. O'Reilly Media. Capítulos 3-5.
2. Date, C. J. (2003). An Introduction to Database Systems. Addison-Wesley. Capítulo 5.
3. Documentación de SQLite: https://www.sqlite.org/lang_select.html
4. SQL Tutorial: <https://www.w3schools.com/sql/>

Criterios de evaluación

Fundamentos de Gestión de Datos					
Rúbrica Holística					
Elemento de la capacidad	Aplica consultas SQL avanzadas con JOINS, agrupaciones, subconsultas y vistas.				
Curso	Fundamentos de Gestión de Datos	Periodo	2026-I		
Actividad	Lab D3 - JOINS y Agrupaciones	Semestre	3		
Nombre del alumno			Semana		
Docente	Pilar Rocío Sayán Mejía	Fecha	Sección		
Criterios a evaluar	Excelente		Bueno	Requiere mejora	Puntaje logrado
1. Revisión de conceptos (Act. 1)	Todas las definiciones correctas con comprensión profunda de JOINS y agrupaciones.		Mayoría correctas con pequeños errores.	Definiciones incompletas o parcialmente correctas.	
2. JOINS (Act. 2)	Aplica correctamente INNER, LEFT, RIGHT JOIN con justificación adecuada.		Aplica JOINS con errores menores.	Aplicación incorrecta o sin justificación.	
3. GROUP BY y HAVING (Act. 2)	Agrupa correctamente y filtra grupos con HAVING, comprende diferencia con WHERE.		Agrupaciones básicas con algunos errores.	No comprende GROUP BY o HAVING.	
4. Subconsultas y vistas (Act. 2)	Crea subconsultas y vistas correctamente, comprende sus ventajas.		Subconsultas básicas o vistas simples.	No aplica subconsultas ni vistas.	
5. Caso de estudio (Act. 3)	TOP 5 correctamente identificado con consultas optimizadas y justificación sólida.		Consultas funcionales con justificación básica.	Consultas incorrectas o sin justificación.	

Total				
--------------	--	--	--	--

Acciones a cumplir	Puntaje
Puntualidad y dedicación	1
Cumplimiento de tiempos establecidos	1
Conclusiones: ortografía y redacción	1
Puntaje total	

Comentarios respecto del desempeño del alumno

Nivel	Descripción
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo la mayoría de requerimientos.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad cumpliendo pocos requerimientos.
No aceptable	No demuestra entendimiento del problema o actividad.